```
# Practical 11: Q-Learning using Grid World

# 1. Import libraries
import numpy as np
import matplotlib.pyplot as plt

# 2. Create Grid World
rows = 4
cols = 4

start_state = 0
goal_state = 15

# Obstacles
obstacles = [5, 9]

# 3. Define actions
# 0 = Up, 1 = Down, 2 = Left, 3 = Right

actions = [0, 1, 2, 3]

# 4. Create Q-table
q_table = np.zeros((rows * cols, len(actions)))

# 5. Set learning parameters
learning_rate = 0.8
discount_factor = 0.95
epsilon = 0.2
episodes = 1000
max_steps = 100
```

```
# 6. Define movement function
def get_next_state(state, action):

    row = state // cols
    col = state % cols

    if action == 0:       # Up
        row = max(0, row - 1)

    elif action == 1:     # Down
        row = min(rows - 1, row + 1)

    elif action == 2:     # Left
        col = max(0, col - 1)

    elif action == 3:     # Right
        col = min(cols - 1, col + 1)

    next_state = row * cols + col

    return next_state

# 7. Train the agent
rewards_per_episode = []

for episode in range(episodes):

    state = start_state
    total_reward = 0

    for step in range(max_steps):
```

```
# Choose action using epsilon-greedy strategy
if np.random.random() < epsilon:
    action = np.random.choice(actions)
else:
    action = np.argmax(q_table[state])

# Get next state
next_state = get_next_state(state, action)

# Calculate reward
if next_state == goal_state:
    reward = 100
    done = True

elif next_state in obstacles:
    reward = -100
    next_state = state
    done = False

else:
    reward = -1
    done = False

# Update Q-table
old_value = q_table[state, action]

next_max = np.max(q_table[next_state])

q_table[state, action] = old_value + learning_rate * (
    reward + discount_factor * next_max - old_value
```

```
        )

        state = next_state
        total_reward += reward

        if done:
            break

    rewards_per_episode.append(total_reward)

# 8. Display Q-table
print("Training Completed!")

print("\nQ-Table:")
print(np.round(q_table, 2))

# 9. Find the best path
state = start_state
path = [state]

for step in range(max_steps):

    action = np.argmax(q_table[state])
    next_state = get_next_state(state, action)

    if next_state in obstacles:
        break

    state = next_state
    path.append(state)
```

```
    if state == goal_state:
        break

print("\nBest Path:", path)

if path[-1] == goal_state:
    print("Goal Reached Successfully!")
else:
    print("Goal Not Reached. Try increasing training episodes.")

# 10. Plot rewards
plt.figure(figsize=(8, 5))

plt.plot(rewards_per_episode)

plt.xlabel("Episodes")
plt.ylabel("Total Reward")
plt.title("Q-Learning Training Progress")
plt.grid(True)
plt.show()

# 11. Visualize the Grid World
grid = np.zeros((rows, cols))

for obstacle in obstacles:
    grid[obstacle // cols, obstacle % cols] = -1

grid[goal_state // cols, goal_state % cols] = 2

for state in path:
    if state != start_state and state != goal_state:
```

```
        grid[state // cols, state % cols] = 1

plt.figure(figsize=(6, 6))
plt.imshow(grid, cmap="viridis")

for i in range(rows):
    for j in range(cols):
        state = i * cols + j

        if state == start_state:
            label = "Start"
        elif state == goal_state:
            label = "Goal"
        elif state in obstacles:
            label = "Wall"
        else:
            label = str(state)

        plt.text(j, i, label, ha="center",
                 va="center", color="white")

plt.title("Q-Learning Grid World")
plt.xticks([])
plt.yticks([])
plt.show()
```